

NOTES

Descriptive Statistics



✓ Descriptive Statistics

✓ Introduction to Statistics

What is Statistics?

Statistics is the science of collecting, organizing, analyzing, and interpreting data to make informed decisions and draw meaningful conclusions. It allows us to understand the world through numbers and patterns, enabling us to quantify uncertainty and variability in real-life scenarios.

Example: Imagine you are conducting a survey to understand public opinion on climate change. Statistics helps you gather data from a sample population, organize it into charts or graphs, analyze trends, and interpret the results to understand the broader public sentiment.

Why is Statistics Important?

Statistics plays a critical role across various domains, helping us make sense of data and solve real-world problems effectively.

1. Understanding Patterns and Trends

Statistics helps in identifying patterns, trends, and correlations within data, making it easier to predict future outcomes or behaviors.

Example 1: Seasonal Sales Trends

A clothing retailer can analyze past sales data to identify trends, such as increased demand for winter coats during November and December. This allows them to stock inventory accordingly and prepare marketing campaigns at the right time.

Example 2: Population Growth

Urban planners use population statistics to predict future housing or infrastructure needs. For instance, if a city's population has been growing at a steady rate, planners can ensure schools and hospitals are built to accommodate the increase.

2. Decision-Making Across Fields

Statistics enables data-driven decision-making in various fields:

Business

Example: A company analyzes customer data to identify their best-selling products and understand customer preferences. For instance, an online retailer notices through sales

data that customers tend to buy more electronics during festive seasons, allowing them to time discounts and promotions effectively.

Sports

Example: Sports teams use player performance data to develop strategies. For instance, a cricket coach might analyze a batter's statistics to determine which types of deliveries they excel against and adjust the batting order accordingly.

Real-World Example

1. Analyze Customer Purchasing Behavior

By examining historical purchase data, companies can group customers based on their buying habits:

Example: A grocery store finds that one group of customers frequently buys organic products while another prefers discounted items. Using these insights, the store can create targeted marketing campaigns or personalized discounts for each group.

2. Measure Product Sales Trends

Statistics helps companies understand how demand for a product changes over time.

Example: A beverage company sees that sales of cold drinks peak during summer and dip during winter. This information helps them focus their advertising efforts and production on warm months to maximize revenue.

3. Inventory Management

Example: A bookstore uses past data to predict demand for popular titles during back-to-school seasons. With accurate forecasts, they avoid understocking or overstocking issues, saving costs and meeting customer demand efficiently.

Types of Statistics

Statistics is broadly divided into two main types:

1. **Descriptive Statistics**
2. **Inferential Statistics**

Together, they form the foundation for analyzing and interpreting data.

Introduction to Descriptive Statistics

Descriptive statistics is a branch of statistics that focuses on summarizing and describing the main features of a dataset. It provides a clear and concise overview of the data, making it easier to identify patterns, trends, and distributions.

Real-World Applications

Business Performance Analysis

A company calculates the average sales per month and visualizes it in a graph to identify peak sales periods.

Sports Performance

The batting average of a cricket player (mean of runs scored per game) provides insights into their consistency.

✓ Types of Descriptive Statistics

1. **Measures of Central Tendency:** Identify the center of the data.
2. **Measures of Variability:** Describe how spread out the data is.
3. **Measures of Shape:** Show the shape and symmetry of the data distribution.

✓ Measures of Central Tendency

These measures identify the "central" or typical value in a dataset, offering insights into its overall behavior.

✓ Mean (Average)

The mean is the sum of all data points divided by the total number of data points. It is sensitive to outliers, as extreme values can significantly skew the result.

$$\text{Mean} = \frac{\text{Sum of all data points}}{\text{Number of data points}}$$

$$\text{Mean} = \frac{\sum x_i}{n}$$

- $\sum$: The summation symbol. It indicates the sum of all values that follow it.
- x_i : Represents the individual data points in the dataset. The subscript i is an index that ranges from 1 to n , representing each data point in turn ($x_1, x_2, x_3, \dots, x_n$).
- n : Represents the total number of data points in the dataset.

Real-World Examples

- **Student Grades:** A teacher calculates the mean test score to gauge the class's overall performance.

Scores: [75, 80, 85, 90, 95]

Mean: $(75+80+85+90+95)/5=85$

- **Household Expenses:** A family calculates the mean monthly expenditure to budget effectively.

Expenses (in \$): [1200, 1300, 1100, 1400, 1500]

Mean: $(1200+1300+1100+1400+1500)/5=1300$

```
import numpy as np
```

```
# Example 1: Student Grades
```

```
scores = [75, 80, 85, 90, 95]
```

```
mean_scores = np.mean(scores)
```

```
print("Student Grades:")
```

```
print("Scores:", scores)
```

```
print("Mean Test Score:", mean_scores)
```

```
# Example 2: Household Expenses
```

```
expenses = [1200, 1300, 1100, 1400, 1500]
```

```
mean_expenses = np.mean(expenses)
```

```
print("\nHousehold Expenses:")
```

```
print("Expenses (in $):", expenses)
```

```
print("Mean Monthly Expenditure:", mean_expenses)
```



```
Student Grades:
```

```
Scores: [75, 80, 85, 90, 95]
```

```
Mean Test Score: 85.0
```

```
Household Expenses:
```

```
Expenses (in $): [1200, 1300, 1100, 1400, 1500]
```

```
Mean Monthly Expenditure: 1300.0
```

✓ Median

The median is the middle value when the data is arranged in sorted order. If the dataset contains an even number of values, the median is the average of the two middle values. The median is less affected by outliers compared to the mean.

Real-World Examples:

- **Income Analysis:** To represent a region's typical income, the median income is often used instead of the mean to avoid distortion by extremely high incomes.

Incomes (in \$): [30,000, 35,000, 40,000, 10,00,000]

Median: [30,000, 35,000, 40,000, 10,00,000] → 35,000 (middle value)

- **Housing Prices:** The median house price provides a more realistic idea of affordability by excluding the influence of luxury property prices.

```
import numpy as np

# Example 1: Income Analysis
incomes = [30000, 35000, 40000, 1000000] # List of incomes
median_income = np.median(incomes)        # Calculate median income

print("Income Analysis:")
print("Incomes (in $):", incomes)
print("Median Income:", median_income)

# Example 2: Housing Prices
housing_prices = [200000, 250000, 300000, 5000000] # List of housing prices
median_price = np.median(housing_prices)           # Calculate median house price

print("\nHousing Prices:")
print("Housing Prices (in $):", housing_prices)
print("Median House Price:", median_price)
```

➡ Income Analysis:
Incomes (in \$): [30000, 35000, 40000, 1000000]
Median Income: 37500.0

Housing Prices:
Housing Prices (in \$): [200000, 250000, 300000, 5000000]
Median House Price: 275000.0

✓ Mode

The mode is the value that appears most frequently in a dataset. Some datasets can have multiple modes (bimodal or multimodal) or no mode at all.

Real-World Examples

- **Retail Analysis:** A store identifies the most popular product size or color based on sales data.
Sales Data: [S, M, M, L, XL, M]
Mode: M (most frequently sold size)
- **Weather Patterns:** Meteorologists analyze the mode of daily temperatures to identify the most common weather pattern in a specific region.

```
import statistics

# Example 1: Retail Analysis
sales_data = ['S', 'M', 'M', 'L', 'XL', 'M'] # List of product sizes sold
mode_size = statistics.mode(sales_data)      # Calculate mode of sales data
```

```
print("Retail Analysis:")
print("Sales Data:", sales_data)
print("Most Popular Size:", mode_size) # Accessing the mode value

# Example 2: Weather Patterns
daily_temperatures = [70, 75, 75, 80, 85, 75, 90] # List of daily temperatures
mode_temperature = statistics.mode(daily_temperatures) # Calculate mode of temperatures

print("\nWeather Patterns:")
print("Daily Temperatures:", daily_temperatures)
print("Most Common Temperature:", mode_temperature) # Accessing the mode value
```

```
➡ Retail Analysis:
Sales Data: ['S', 'M', 'M', 'L', 'XL', 'M']
Most Popular Size: M

Weather Patterns:
Daily Temperatures: [70, 75, 75, 80, 85, 75, 90]
Most Common Temperature: 75
```

```
import numpy as np
import statistics

# Example datasets
dataset1 = [10, 20, 30, 40, 50, 100]
dataset2 = [10, 20, 30, 30, 40, 50]

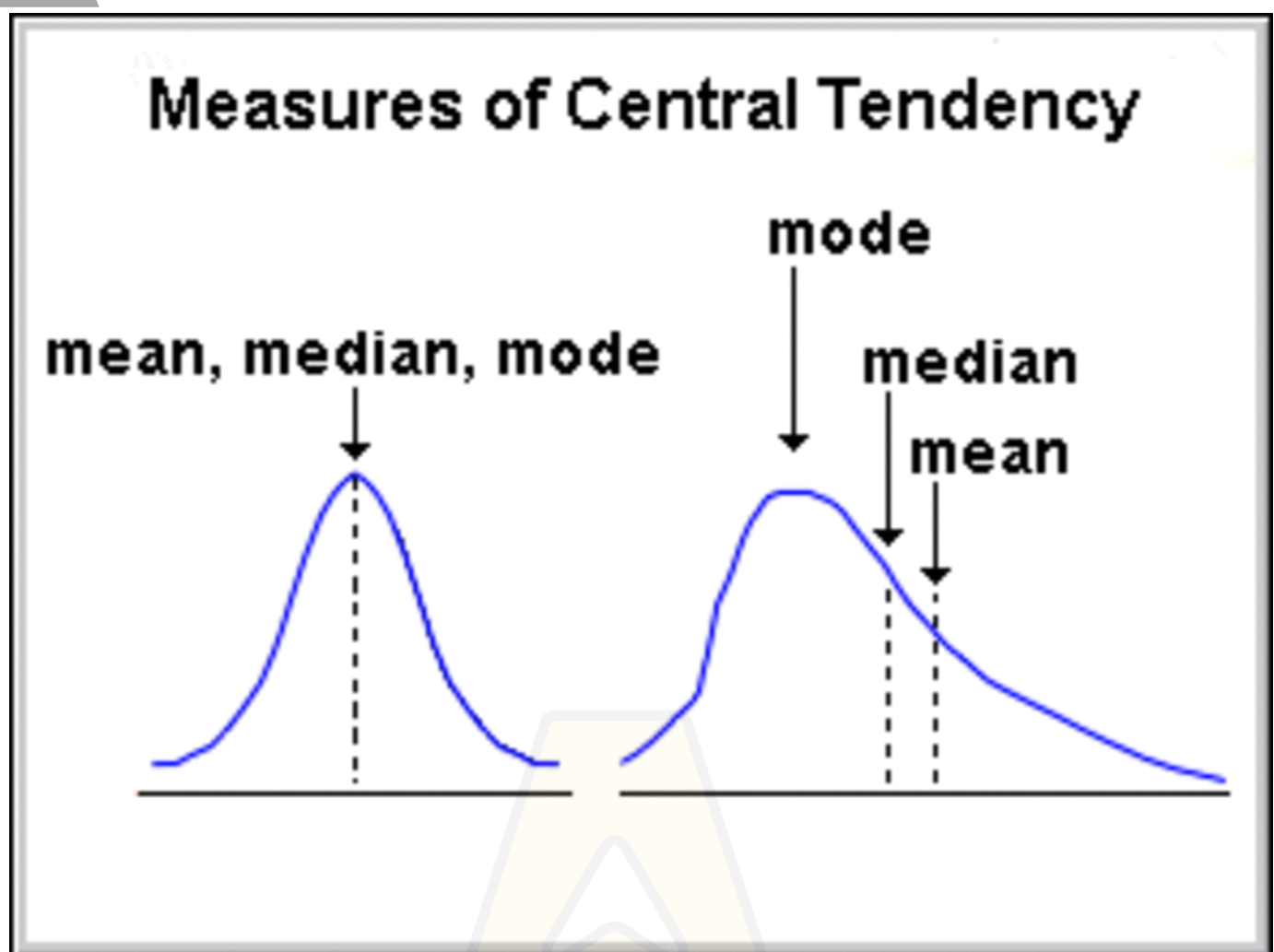
# Function to calculate and print statistics
def calculate_statistics(data):
    mean = np.mean(data)
    median = np.median(data)
    mode = statistics.mode(data)

    print(f"Dataset: {data}")
    print(f"Mean: {mean:.2f}")
    print(f"Median: {median}")
    print(f"Mode: {mode}\n")

# Calculate statistics for each dataset
calculate_statistics(dataset1)
calculate_statistics(dataset2)
```

```
➡ Dataset: [10, 20, 30, 40, 50, 100]
Mean: 41.67
Median: 35.0
Mode: 10

Dataset: [10, 20, 30, 30, 40, 50]
Mean: 30.00
Median: 30.0
Mode: 30
```



Measures of Variability

Measures of variability describe the spread or dispersion of data, giving insight into how values differ from one another and the central tendency (e.g., mean). A dataset's variability helps assess data consistency and reliability.

Range

The range is the simplest measure of variability, calculated as the difference between the maximum and minimum values in a dataset.

$$\text{Range} = \text{Maximum Value} - \text{Minimum Value}$$

Real-World Examples

- Temperature Variations:** The range of daily temperatures helps us understand how much the temperature fluctuates during a week.

Example: [15°C, 20°C, 25°C, 30°C, 35°C] → Range = 35°C - 15°C = 20°C

- **Stock Market:** The range of stock prices during a day provides insight into market volatility.

```
import numpy as np

# Example datasets
temperatures = [15, 20, 25, 30, 35] # Daily temperatures in °C
stock_prices = [100, 150, 120, 130, 200] # Stock prices in $

# Function to calculate and print the range
def calculate_range(data):
    range_value = np.max(data) - np.min(data)
    return range_value

# Calculate range for temperatures
temperature_range = calculate_range(temperatures)
print(f"Temperature Variations:")
print(f"Daily Temperatures: {temperatures}")
print(f"Range: {temperature_range}°C\n")

# Calculate range for stock prices
stock_price_range = calculate_range(stock_prices)
print(f"Stock Market:")
print(f"Stock Prices: {stock_prices}")
print(f"Range: ${stock_price_range}")
```

```
➡ Temperature Variations:
Daily Temperatures: [15, 20, 25, 30, 35]
Range: 20°C

Stock Market:
Stock Prices: [100, 150, 120, 130, 200]
Range: $100
```

✓ Variance

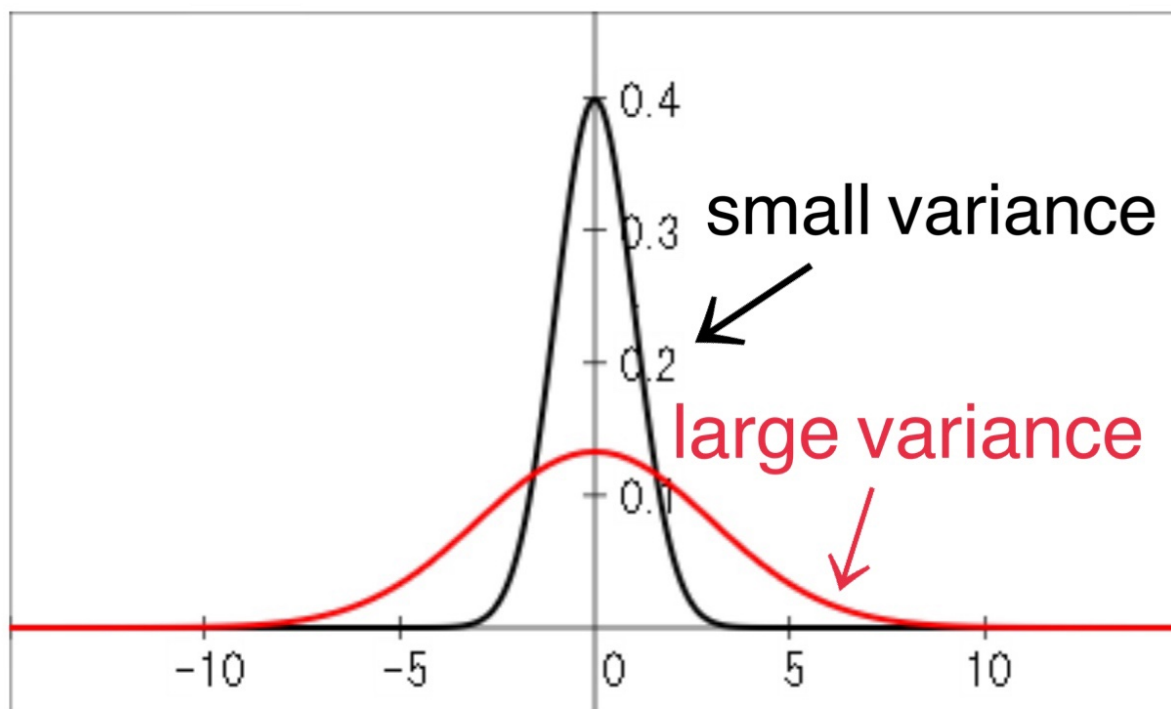
Variance measures the average squared deviations of each data point from the mean, giving a sense of the data's spread. Larger variance indicates greater dispersion.

The formula for variance is:

$$\text{Variance} = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

Where:

- x_i : Each individual data point in the dataset.
- $\bar{x}$: The mean (average) of the data.
- n : The total number of data points in the dataset.



Real-World Examples

- **Exam Scores:** Variance in student scores helps determine if the performance is consistent or widely spread.
Example: Scores = [70, 72, 74, 90, 95] → High variance due to the outliers (90, 95).
- **Manufacturing Quality:** Variance in product dimensions indicates consistency in the production process.

```
import numpy as np

def calculate_variance(data):
    return np.var(data, ddof=0) # Population variance (ddof=0)

# Example 1: Exam Scores
exam_scores = [70, 72, 74, 90, 95]
exam_variance = calculate_variance(exam_scores)

# Example 2: Manufacturing Quality
# Let's assume we have product dimensions in millimeters
product_dimensions = [100.0, 100.5, 99.8, 101.2, 102.0]
quality_variance = calculate_variance(product_dimensions)

# Print results
print(f"Exam Scores: {exam_scores}")
print(f"Variance in Exam Scores: {exam_variance:.2f} (High variance due to outliers)")

print(f"\nProduct Dimensions: {product_dimensions}")
```

```
print(f"Variance in Product Dimensions: {quality_variance:.2f}" (Indicates consistency in
```

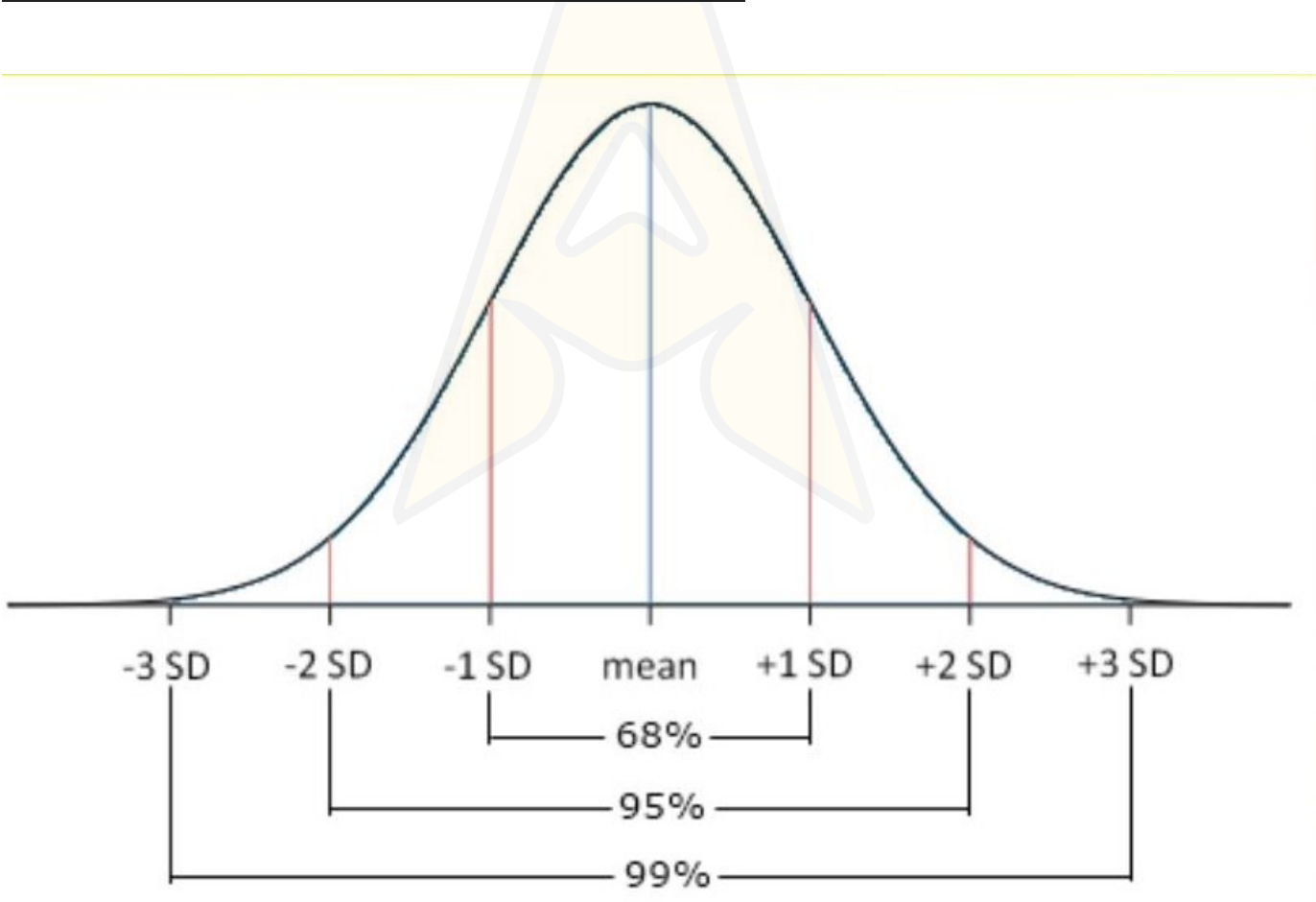
```
➡ Exam Scores: [70, 72, 74, 90, 95]  
Variance in Exam Scores: 104.96 (High variance due to outliers)
```

```
Product Dimensions: [100.0, 100.5, 99.8, 101.2, 102.0]  
Variance in Product Dimensions: 0.66 (Indicates consistency in production)
```

✓ **Standard Deviation (SD)**

The standard deviation is the square root of the variance, offering a more interpretable measure of data dispersion in the same units as the data. In simple words, standard deviation measures how spread out the data is around the mean (average). It tells us whether the data points are close to the mean or scattered far away.

Standard Deviation = $\sqrt{\text{Variance}}$



Real-World Examples

- **Sports Performance:** A cricket player with a low standard deviation in their scores shows consistent performance.

Example: Scores = [45, 46, 44, 45] → Low SD = Consistent Performance.

- **Financial Investments:** A stock with a high standard deviation in returns indicates higher risk.

```
import numpy as np

# Example datasets
cricket_scores = [45, 46, 44, 45] # Scores of a cricket player
stock_returns = [5, 10, -2, 15, -5] # Returns of a stock investment

# Function to calculate and print standard deviation
def calculate_standard_deviation(data):
    std_dev = np.std(data)
    return std_dev

# Calculate standard deviation for cricket scores
cricket_std_dev = calculate_standard_deviation(cricket_scores)
print("Sports Performance:")
print(f"Cricket Scores: {cricket_scores}")
print(f"Standard Deviation: {cricket_std_dev:.2f} (Low SD = Consistent Performance)\n")

# Calculate standard deviation for stock returns
stock_std_dev = calculate_standard_deviation(stock_returns)
print("Financial Investments:")
print(f"Stock Returns: {stock_returns}")
print(f"Standard Deviation: {stock_std_dev:.2f} (High SD = Higher Risk)")
```

⇒ Sports Performance:
Cricket Scores: [45, 46, 44, 45]
Standard Deviation: 0.71 (Low SD = Consistent Performance)

Financial Investments:
Stock Returns: [5, 10, -2, 15, -5]
Standard Deviation: 7.39 (High SD = Higher Risk)

✓ Interquartile Range (IQR)

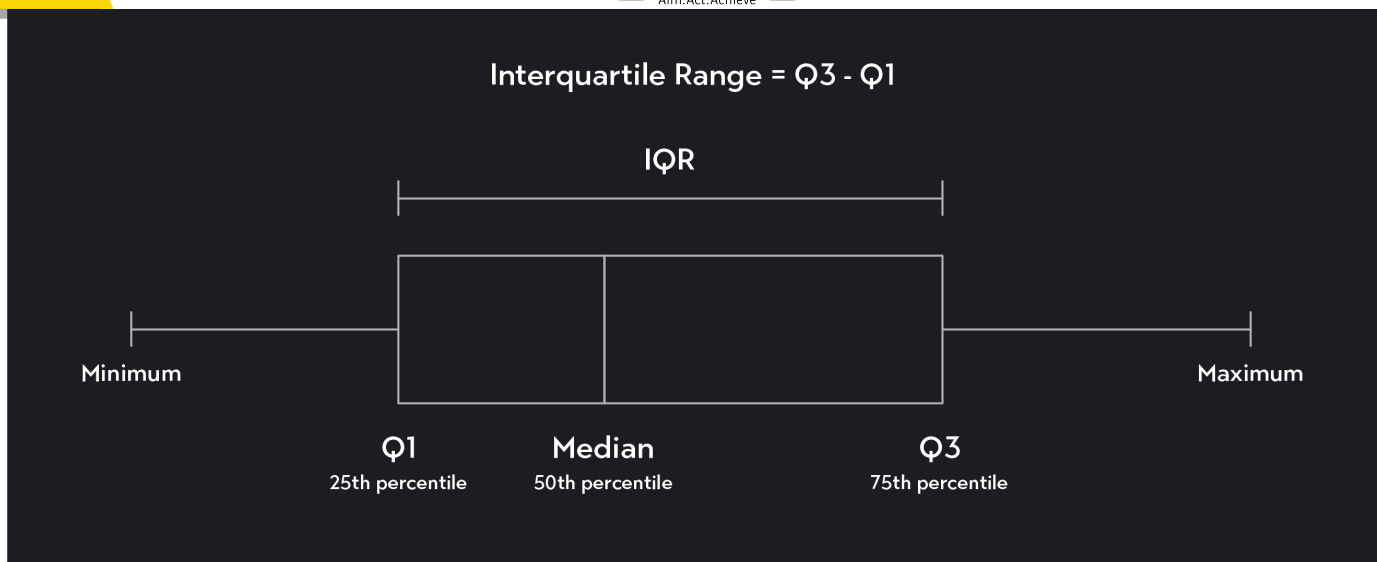
The IQR measures the range within which the central 50% of data lies, offering a robust measure of variability unaffected by outliers.

The formula for the Interquartile Range (IQR) is:

$$IQR = Q_3 - Q_1$$

Where:

- Q_1 : The 25th percentile (lower quartile).
- Q_3 : The 75th percentile (upper quartile).



Steps to Calculate the Interquartile Range (IQR):

1. Arrange the data in ascending order.
2. Identify Q_1 (the median of the lower half of the data).
3. Identify Q_3 (the median of the upper half of the data).
4. Subtract Q_1 from Q_3 .

Real-World Examples

- **Income Distribution:** The IQR helps identify income inequality by focusing on the middle range of salaries.
Example: [25,000, 30,000, 35,000, 40,000, 200,000] → IQR excludes the outlier (200,000).
- **Delivery Times:** The IQR of delivery times helps measure service consistency by ignoring unusually fast or slow deliveries.

```
import numpy as np

# Example datasets
income_distribution = [25000, 30000, 35000, 40000, 200000] # Income values
delivery_times = [30, 45, 50, 60, 120] # Delivery times in minutes

# Function to calculate IQR
def calculate_iqr(data):
    q1 = np.percentile(data, 25) # First quartile (25th percentile)
    q3 = np.percentile(data, 75) # Third quartile (75th percentile)
    iqr = q3 - q1 # Interquartile range
    return iqr

# Calculate IQR for income distribution
income_iqr = calculate_iqr(income_distribution)
print("Income Distribution:")
print(f"Income Values: {income_distribution}")
print(f"IQR: {income_iqr} (Excludes outlier)\n")
```

```
# Calculate IQR for delivery times
delivery_iqr = calculate_iqr(delivery_times)
print("Delivery Times:")
print(f"Delivery Times (minutes): {delivery_times}")
print(f"IQR: {delivery_iqr} (Measures service consistency)")
```

➡ Income Distribution:
Income Values: [25000, 30000, 35000, 40000, 200000]
IQR: 10000.0 (Excludes outlier)

Delivery Times:
Delivery Times (minutes): [30, 45, 50, 60, 120]
IQR: 15.0 (Measures service consistency)

✓ Measures of Shape (Skewness and Kurtosis)

The measures of shape provide insights into the distribution's symmetry and the presence of outliers. These are crucial for understanding data characteristics and choosing the right analytical techniques.

✓ Skewness

Skewness quantifies the asymmetry of a data distribution. A perfectly symmetrical distribution (e.g., normal distribution) has a skewness of zero. Skewness helps identify whether data leans more to the right or left of the mean.

The formula for sample skewness is:

$$\text{Skewness} = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$$

Where:

- n : Number of observations.
- x_i : Individual data point.
- $\bar{x}$: Mean of the data.
- σ : Standard deviation of the data.

```
import numpy as np
```

```
def skew(distribution):
    n = len(distribution)
    mean = np.mean(distribution)
    std = np.std(distribution)

    # Divide the formula into two parts
    first_part = n / ((n - 1) * (n - 2))
    second_part = np.sum(((distribution - mean) / std) ** 3)

    skewness = first_part * second_part
```

return skewness

```
# Example dataset: Simple integers
simple_data = [1, 2, 2, 3, 4, 10] # A small dataset with a potential outlier

# Calculate skewness using the moment-based method
skewness_result = skew(simple_data)

# Print the result
print(f"Skewness score of the distribution is {skewness_result:.5f}")
```

➡ Skewness score of the distribution is 2.55331

$$\text{Skewness} = \frac{3(\text{mean} - \text{median})}{\text{Standard deviation}}$$

```
import numpy as np
import pandas as pd

# Example dataset: Prices of fruits
fruit_prices = pd.Series([2.5, 3.0, 2.8, 3.5, 10.0]) # Prices in dollars

# Calculate mean, median, and standard deviation
mean_price = fruit_prices.mean()
median_price = fruit_prices.median()
std = fruit_prices.std()

# Calculate Pearson's second skewness score
skewness = (3 * (mean_price - median_price)) / std

# Print the result
print(f"Skewness score of the fruit prices distribution is {skewness:.5f}")
```

➡ Skewness score of the fruit prices distribution is 1.28553

Types of Skewness:

1. Positive Skew (Right Skew)
2. Negative Skew (Left Skew)

✓ Positive Skew (Right Skew)

The tail extends more to the right, indicating higher values.

Example: Income Data

Most people earn moderate incomes, but a few earn very high salaries, creating a long right tail.

```
from scipy.stats import skew
```

```
# Positive Skew Example: Income Data
income_data = [10, 20, 30, 40, 100] # Positive skew due to the high value
positive_skewness = skew(income_data)
print("Positive Skew (Right Skew):")
print(f"Income Data: {income_data}")
print(f"Skewness: {positive_skewness:.2f} (Right tail extends)\n")
```

```
➡ Positive Skew (Right Skew):
Income Data: [10, 20, 30, 40, 100]
Skewness: 1.14 (Right tail extends)
```

✓ Negative Skew (Left Skew)

The tail extends more to the left, indicating lower values.

Example: Age at Retirement

Most people retire around a typical age, but some retire much earlier, creating a long left tail.

```
from scipy.stats import skew
```

```
# Negative Skew Example: Age at Retirement
retirement_data = [10, 20, 30, 40, 15] # Negative skew due to the low value
negative_skewness = skew(retirement_data)
print("Negative Skew (Left Skew):")
print(f"Retirement Ages: {retirement_data}")
print(f"Skewness: {negative_skewness:.2f} (Left tail extends)")
```

```
➡ Negative Skew (Left Skew):
Retirement Ages: [10, 20, 30, 40, 15]
Skewness: 0.40 (Left tail extends)
```

Real-Life Examples of Skewness

1. Sales Data:

- Description: Seasonal sales spikes during holidays (e.g., Christmas, Black Friday) create a positive skew. Most sales occur during these specific times, leading to a distribution where a few days have extremely high sales figures compared to the rest.
- Hypothetical Data: [100, 150, 200, 3000] (where 3000 represents a holiday spike).

2. Test Scores:

- Description: If a test is very easy, most students score high, but a few lower scores might create a negative skew. This means that while the majority of scores are clustered at the high end, there are some lower scores that pull the average down.
- Hypothetical Data: [90, 95, 98, 100, 50] (where 50 is an unusually low score).


```
from scipy.stats import skew

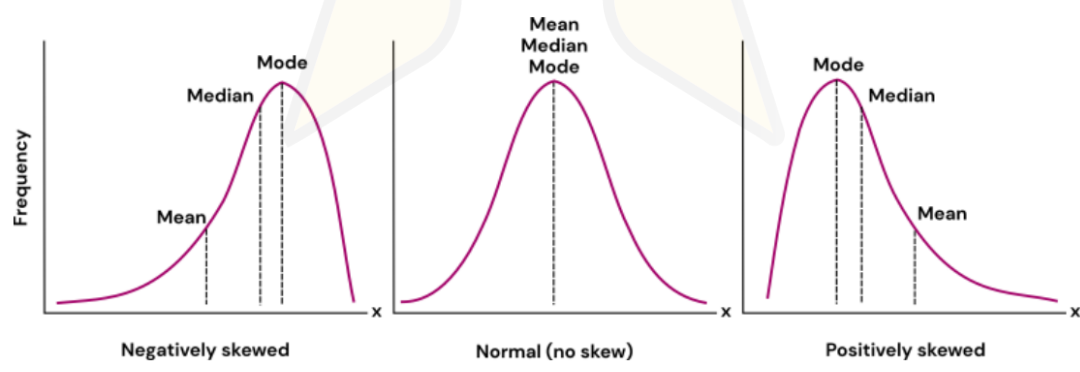
# Hypothetical datasets
sales_data = [100, 150, 200, 3000]      # Positive skew due to holiday spike
test_scores = [90, 95, 98, 100, 50]     # Negative skew due to low score

# Function to calculate and print skewness
def print_skewness(data, description):
    calculated_skewness = skew(data)
    print(f"{description}:")
    print(f"Data: {data}")
    print(f"Skewness: {calculated_skewness:.2f}\n")

# Calculate and print skewness for each example
print_skewness(sales_data, "Sales Data (Positive Skew)")
print_skewness(test_scores, "Test Scores (Negative Skew)")
```

```
➡ Sales Data (Positive Skew):
   Data: [100, 150, 200, 3000]
   Skewness: 1.15

   Test Scores (Negative Skew):
   Data: [90, 95, 98, 100, 50]
   Skewness: -1.38
```



✓ **Kurtosis**

Kurtosis measures the "tailedness" of the data distribution and identifies how extreme values (outliers) affect the dataset.

$$Kurtosis = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \text{mean}}{\text{Stdev}} \right)^4 - 3$$

```
import numpy as np

def kurtosis(distribution):
    n = len(distribution)
    mean = np.mean(distribution)
    std = np.std(distribution)

    # Calculate kurtosis
    kurtosis = (1 / n) * np.sum(((distribution - mean) / std) ** 4) - 3

    return kurtosis

simple_data = [1, 2, 2, 3, 4, 10] # A small dataset with a potential outlier

# Calculate kurtosis using the moment-based method
kurtosis_result = kurtosis(simple_data)

# Print the result
print(f"Kurtosis score of the distribution is {kurtosis_result:.5f}")
```

⇒ Kurtosis score of the distribution is 0.53344

Excess Kurtosis

Excess kurtosis is the measure of kurtosis relative to a normal distribution. For a normal distribution, the kurtosis is exactly 3. Excess kurtosis adjusts this by subtracting 3, making it easier to interpret whether the distribution has heavier or lighter tails compared to a normal curve.

Excess Kurtosis = Kurtosis - 3

The formula for sample kurtosis (specifically, excess kurtosis) is:

$$Kurtosis = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

Where:

- n : Number of observations.
- x_i : Individual data point.
- $\bar{x}$: Mean of the data.
- σ : Standard deviation of the data.

1. High Kurtosis (kurtosis > 3, excess kurtosis > 0) – Leptokurtic

The distribution has heavy tails, meaning there are more extreme values (outliers) compared to a normal distribution.

Example: Stock market returns often exhibit high kurtosis, indicating large fluctuations (extreme gains or losses).

2. Normal Kurtosis (kurtosis = 3, excess kurtosis = 0) – Mesokurtic

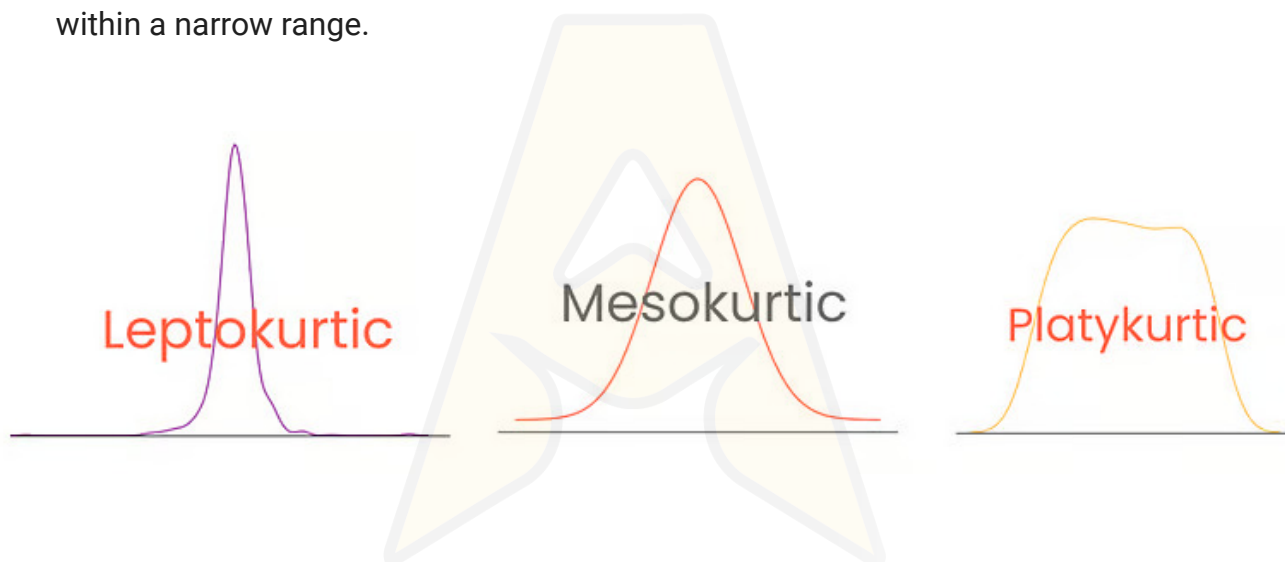
The distribution resembles a normal bell curve with moderate tails and a typical number of outliers.

Example: Heights of people in a population often follow a mesokurtic distribution.

3. Low Kurtosis (kurtosis < 3, excess kurtosis < 0) – Platykurtic

The distribution has light tails, meaning there are fewer extreme values (outliers).

Example: Uniformly distributed test scores often have low kurtosis, as most values cluster within a narrow range.



```
import numpy as np
```

```
def kurtosis(distribution):  
    n = len(distribution)  
    mean = np.mean(distribution)  
    std = np.std(distribution)  
  
    # Calculate kurtosis (without subtracting 3)  
    kurtosis = (1 / n) * np.sum(((distribution - mean) / std) ** 4)  
  
    # Return excess kurtosis by subtracting 3  
    return kurtosis - 3  
  
# 1. Leptokurtic: High kurtosis (heavy tails)  
# Example: Stock market returns with extreme fluctuations  
leptokurtic_data = [1, 1, 2, 2, 3, 10, 15, 30] # More extreme values  
leptokurtic_kurtosis = kurtosis(leptokurtic_data)  
  
# 2. Mesokurtic: Normal kurtosis (bell curve)  
# Example: Heights of people in a population
```

```
mesokurtic_data = [150, 152, 154, 155, 156, 158, 160] # Values clustered around the mean  
mesokurtic_kurtosis = kurtosis(mesokurtic_data)
```

```
# 3. Platykurtic: Low kurtosis (light tails)
```

```
# Example: Uniformly distributed test scores
```

```
platykurtic_data = [50, 55, 60, 65, 70, 75, 80, 85] # Few extreme values
```

```
platykurtic_kurtosis = kurtosis(platykurtic_data)
```

```
# Print results
```

```
print(f"Leptokurtic Data Kurtosis: {leptokurtic_kurtosis:.5f} (Excess Kurtosis, High)")
```

```
print(f"Mesokurtic Data Kurtosis: {mesokurtic_kurtosis:.5f} (Excess Kurtosis, Normal)")
```

```
print(f"Platykurtic Data Kurtosis: {platykurtic_kurtosis:.5f} (Excess Kurtosis, Low)")
```



```
Leptokurtic Data Kurtosis: 0.65332 (Excess Kurtosis, High)
```

```
Mesokurtic Data Kurtosis: -0.98000 (Excess Kurtosis, Normal)
```



THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)